Metacharacters control things like quantifying (e.g., the Kleene star), grouping, counting, and even capturing parts of the text being searched. Let's start with some simple examples.

```
$text = 'Highly irregular was the time I found a human foot in a
toaster oven';
$pattern = '/toaster/';
var_dump(preg_match($pattern, $text));
```

```
Output:
int(1)
```

Here we've used a core string function – `preg_match()` – to test that `$pattern` (as the first argument) can be successfully applied to `$text` (the second argument), and the function returns `int(1)` to indicate that the pattern exists (rather than, say, a boolean – by now you should have a feel for the inconsistencies in PHP string functions). The pattern itself – `/toaster/` is very simple – characters in a regular expression represent themselves unless modified with a metacharacter – and that particular string does indeed exist in the search text. (Note that this is an inefficient way to match simple substrings, `strpos()` is much faster.)

There's a lot more to `preg_match()`, but we'll save the fine detail for when cover the various code and `mbstring` PCRE functions at the end of the section. Before then, let's go deep on the syntax and features of PCREs.

Delimiters

The patterns supplied to all the `preg_*` functions must be enclosed in matching delimiter characters. The convention is to use `/`, but other characters are available. Specifically, these are any non-alphanumeric, non-backslash, non-whitespace ASCII character. Hash symbols # and percentage signs % are common alternatives; the recommendation is to stick with forward slash unless it appears as a common character in the pattern (because it will make for a more difficult pattern to read, as you'll see in a moment).

```
var_dump(preg_match('  #toaster#', $text));
var_dump(preg_match('+toaster+', $text));
var_dump(preg_match('&toaster&', $text));
```